

Heart Disease Dataset

This data set dates from 1988 and consists of four databases: Cleveland, Hungary, Switzerland, and Long Beach V. It contains 76 attributes, including the predicted attribute, but all published experiments refer to using a subset of 14 of them. The "target" field refers to the presence of heart disease in the patient. It is integer valued 0 = no disease and 1 = disease.

Attribute Information:

- age - age
- sex - sex
- cp - chest pain type (4 values)
- trestbps - resting blood pressure
- chol - serum cholestoral in mg/dl
- fbs - fasting blood sugar > 120 mg/dl
- restecg - resting electrocardiographic results (values 0,1,2)
- thalach - maximum heart rate achieved
- exang - exercise induced angina
- oldpeak - ST depression induced by exercise relative to rest
- slope - the slope of the peak exercise ST segment
- ca - number of major vessels (0-3) colored by flourosopy
- thal - 0 = normal; 1 = fixed defect; 2 = reversable defect
- target

```
In [1]: # Importing Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings("ignore")
```

```
In [2]: # Load the dataset
data = pd.read_csv("C:\\Users\\rache\\OneDrive\\Rachel Study Materials\\Project Dataset\\heart_disease_dataset.csv")
```

```
In [3]: # First Five rows of the dataset
data.head()
```

```
Out[3]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	1
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	1
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	1
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	1

```
In [4]: #Shape of the dataset
data.shape
```

```
Out[4]: (303, 14)
```

```
In [5]: # datatypes
data.dtypes
```

```
Out[5]: age          int64
sex          int64
cp          int64
trestbps    int64
chol        int64
fbs         int64
restecg     int64
thalach     int64
exang       int64
oldpeak     float64
slope       int64
ca          int64
thal        int64
target      int64
dtype: object
```

```
In [6]: # Information of the Dataset
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         303 non-null   int64
1   sex         303 non-null   int64
2   cp          303 non-null   int64
3   trestbps    303 non-null   int64
4   chol        303 non-null   int64
5   fbs         303 non-null   int64
6   restecg     303 non-null   int64
7   thalach     303 non-null   int64
8   exang       303 non-null   int64
9   oldpeak     303 non-null   float64
10  slope       303 non-null   int64
11  ca          303 non-null   int64
12  thal        303 non-null   int64
13  target      303 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
In [7]: # Summary of the dataset
data.describe()
```

```
Out[7]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053	149.646865	0.326733
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860	22.905161	0.469794
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000	133.500000	0.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000	153.000000	0.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000	166.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000

```
In [8]: # Null values
data.isnull().sum()
```

```
Out[8]: age          0
sex          0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

- There are no null values or missing values from the summary of the dataset.

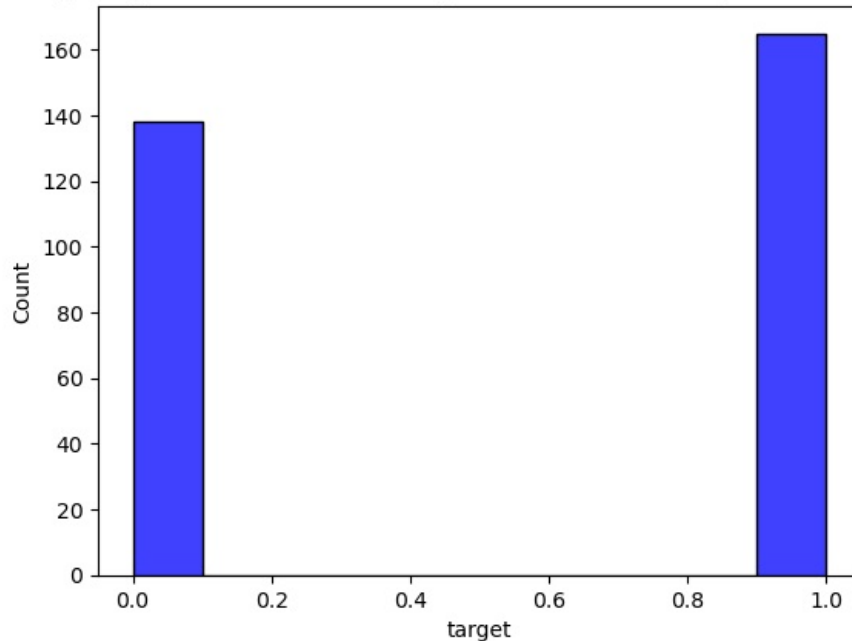
- There is some skewness in trestbps, chol, thalach.

EDA

```
In [9]: # Histogram
plt.title("Histogram plot for the count of target variable how many has heart disease")
sns.histplot(data["target"], color = "b")
```

```
Out[9]: <Axes: title={'center': 'Histogram plot for the count of target variable how many has heart disease'}, xlabel='target', ylabel='Count'>
```

Histogram plot for the count of target variable how many has heart disease



- Around **170** patients has heart disease and 140 people has no heart disease.

```
In [10]: #Scatterplot
plt.title("Patients Age and their chest pain levels differ from gender")
sns.scatterplot(x = 'age', y = 'cp', data = data, hue = 'sex')
```

```
Out[10]: <Axes: title={'center': 'Patients Age and their chest pain levels differ from gender'}, xlabel='age', ylabel='cp'>
```



- Most Patients suffer from chest pain are Male compared to Female.
- Their Age from around **40 to 70**.
- Their chest pain level around 2.0.

```
In [11]: sns.pairplot(data)
```

```
Out[11]: <seaborn.axisgrid.PairGrid at 0x1f324a4fe00>
```

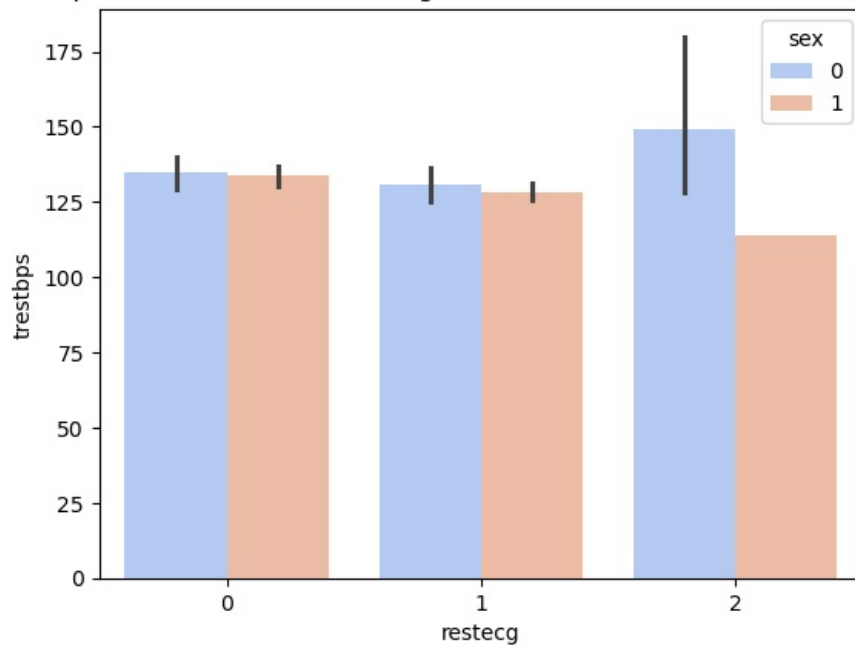


- Firstly we notice most of the variables has discrete values.
- Also we observe the data points are not normal distributed, which clearly shows skewness and outliers.

```
In [12]: # Barplot
plt.title("Barplot of Rest ECG and resting Blood Pressure difference with Gender")
sns.barplot(x = 'restecg', y = 'trestbps', data = data, hue = 'sex', palette = 'coolwarm')
```

```
Out[12]: <Axes: title={'center': 'Barplot of Rest ECG and resting Blood Pressure difference with Gender'}, xlabel='restecg', ylabel='trestbps'>
```

Barplot of Rest ECG and resting Blood Pressure difference with Gender

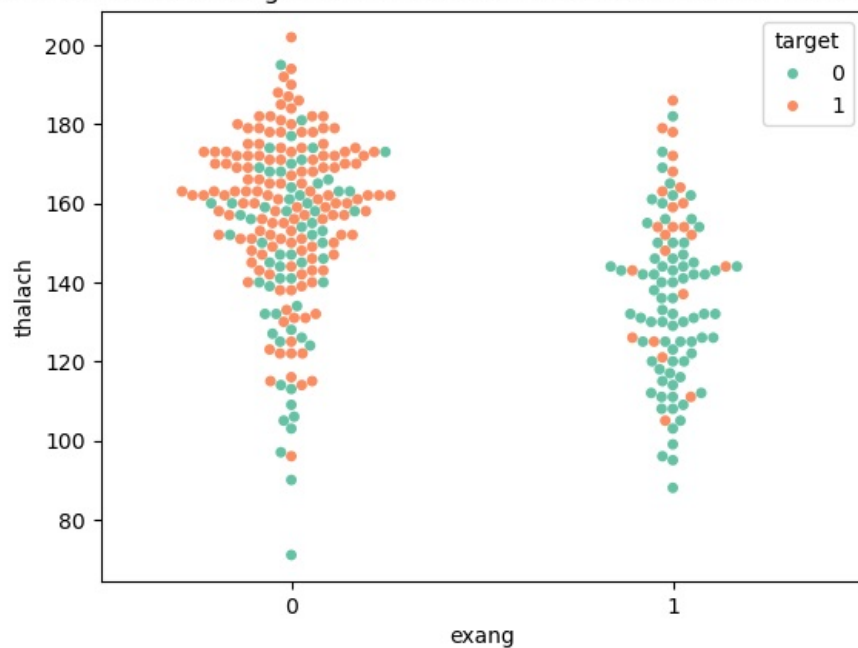


- From the barplot resting ECG results is higher in Female compared to male around 150 resting blood Pressure.
- In male RestECG is high in 0 and 1 results compared to 2.

```
In [13]: #swarmplot
plt.title("Swarmplot of exercise induced Angina and maximum heart rate achieved with Heart Disease Patients")
sns.swarmplot(x='exang', y='thalach', data=data, hue = "target", palette = 'Set2')
```

```
Out[13]: <Axes: title={'center': 'Swarmplot of exercise induced Angina and maximum heart rate achieved with Heart Disease Patients'}, xlabel='exang', ylabel='thalach'>
```

Swarmplot of exercise induced Angina and maximum heart rate achieved with Heart Disease Patients



- Maximum heart rate achieved with Heart patients with no exercise induced Angina are around 160 and 175.
- With exercise induced Angina mostly are not heart patients.

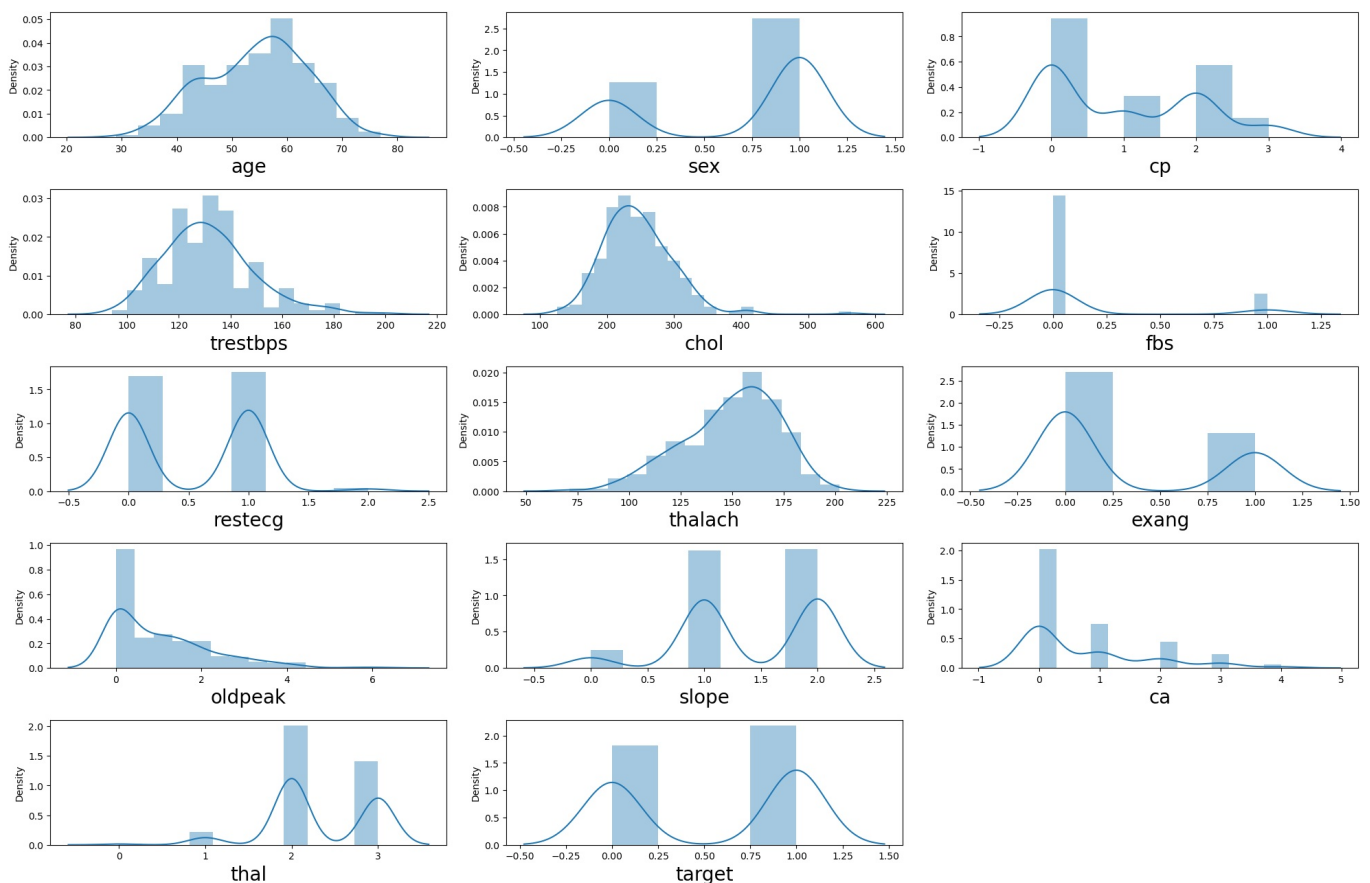
Data Preprocessing

```
In [14]: # Lets check the skewness
data.skew()
```

```
Out[14]: age      -0.202463
sex        -0.791335
cp         0.484732
trestbps   0.713768
chol       1.143401
fbs        1.986652
restecg    0.162522
thalach    -0.537410
exang      0.742532
oldpeak    1.269720
slope     -0.508316
ca         1.310422
thal       -0.476722
target     -0.179821
dtype: float64
```

- From the Continuous variables skewness **resting blood pressure(trestbps)** is **moderate right skew**.
- **Cholesterol (chol)** is **right skewed**
- **maximum heart rate achieved(thalach)** is **Mild left skew**
- **oldpeak** is **high right skew**

```
In [15]: #Checking the Skewness
plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
    if plotnumber<=14:
        ax=plt.subplot(6,3,plotnumber)
        sns.distplot(data[column])
        plt.xlabel(column,fontsize=20)
        plotnumber +=1
plt.tight_layout()
```

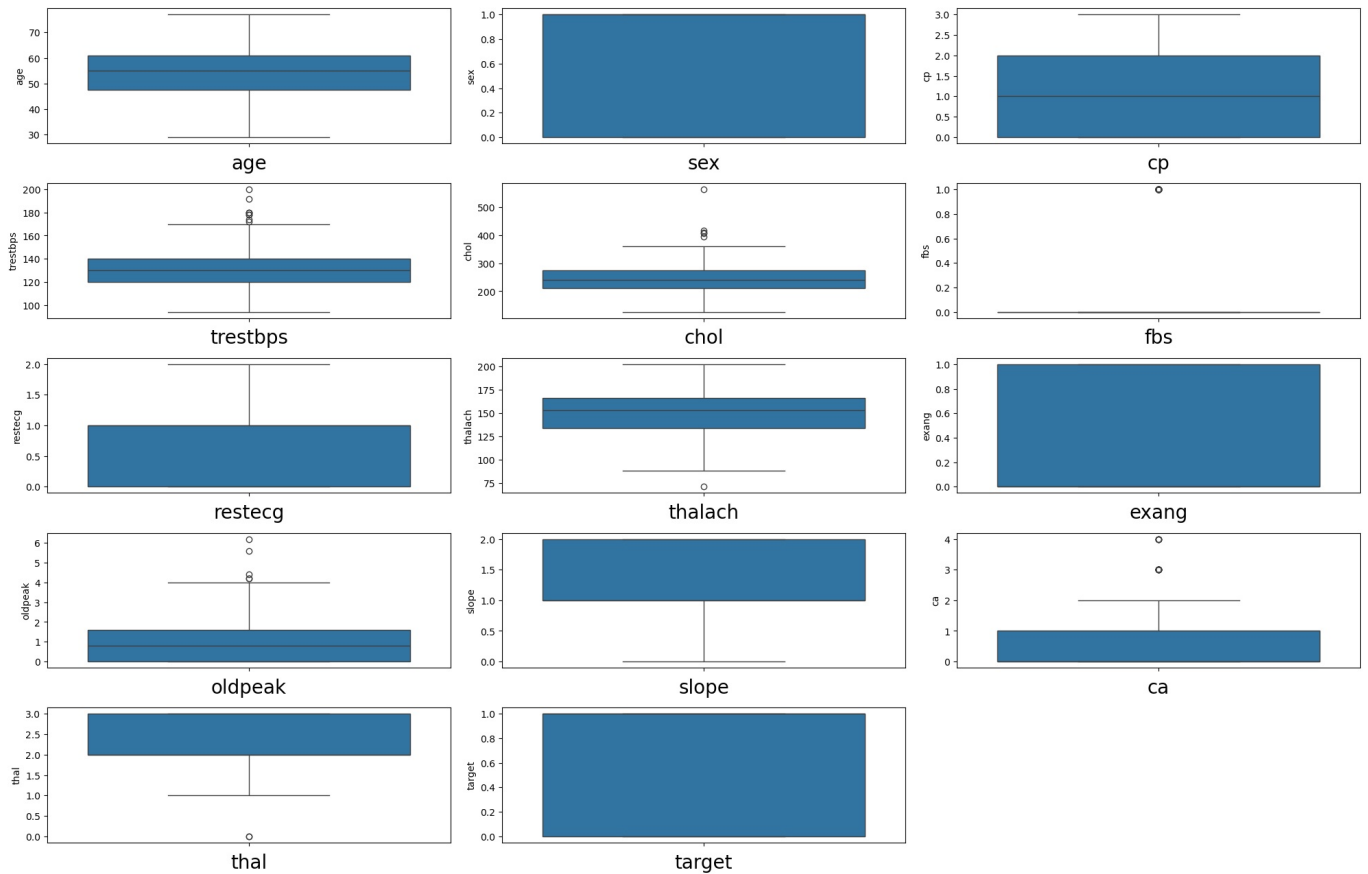


```
In [16]: # Check the outliers
plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
```

```

if plotnumber<=14:
    ax=plt.subplot(6,3,plotnumber)
    sns.boxplot(data[column])
    plt.xlabel(column,fontsize=20)
    plotnumber +=1
plt.tight_layout()

```



- From visualization we clearly understand trestbps, chol, thalach, oldpeak needs to be treated.
- We skip thalach since it is mild left skew.

```

In [17]: # Treat the outliers and skewness
#1st Quantile
q1 = data.quantile(0.25)

```

```

In [18]: #3rd Quantile
q3 = data.quantile(0.75)

```

```

In [19]: # IQR Range
IQR = q3 - q1

```

```

In [20]: print("1st Quantile:")
print(q1)
print(" ")
print("3rd Quantile:")
print(q3)
print(" ")
print("Inter Quartile Range:")
print(IQR)

```

```
1st Quantile:
age          47.5
sex          0.0
cp           0.0
trestbps     120.0
chol         211.0
fbs          0.0
restecg      0.0
thalach      133.5
exang        0.0
oldpeak      0.0
slope        1.0
ca           0.0
thal         2.0
target       0.0
Name: 0.25, dtype: float64
```

```
3rd Quantile:
age          61.0
sex          1.0
cp           2.0
trestbps     140.0
chol         274.5
fbs          0.0
restecg      1.0
thalach      166.0
exang        1.0
oldpeak      1.6
slope        2.0
ca           1.0
thal         3.0
target       1.0
Name: 0.75, dtype: float64
```

```
Inter Quartile Range:
age          13.5
sex          1.0
cp           2.0
trestbps     20.0
chol         63.5
fbs          0.0
restecg      1.0
thalach      32.5
exang        1.0
oldpeak      1.6
slope        1.0
ca           1.0
thal         1.0
target       1.0
dtype: float64
```

```
In [21]: ### Outliers Detection Formula
```

```
##### Upper Limit: Q3 +(1.5 * IQR)
```

```
##### Lower Limit: Q1 -(1.5 * IQR)
```

```
In [22]: #Validating Resting Blood pressure outlier
```

```
trest_pred_high = (q3.trestbps + (1.5 * IQR.trestbps))
trest_pred_high
```

```
Out[22]: np.float64(170.0)
```

```
In [23]: #Check the indexes which have higher values
```

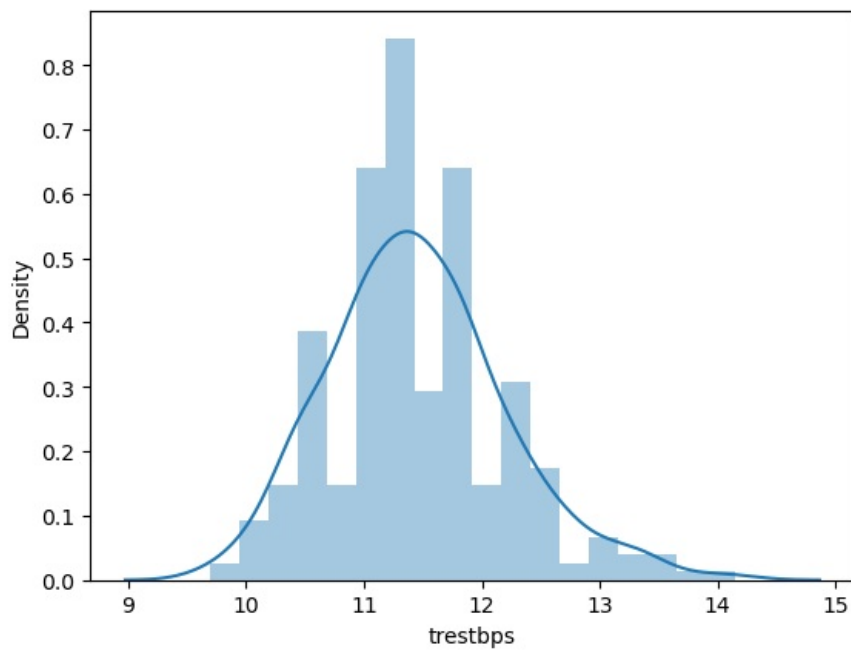
```
index=np.where(data['trestbps']>trest_pred_high)
index
```

```
Out[23]: (array([ 8, 101, 110, 203, 223, 241, 248, 260, 266]),)
```

```
In [24]: data['trestbps'] = np.sqrt(data['trestbps'])
```

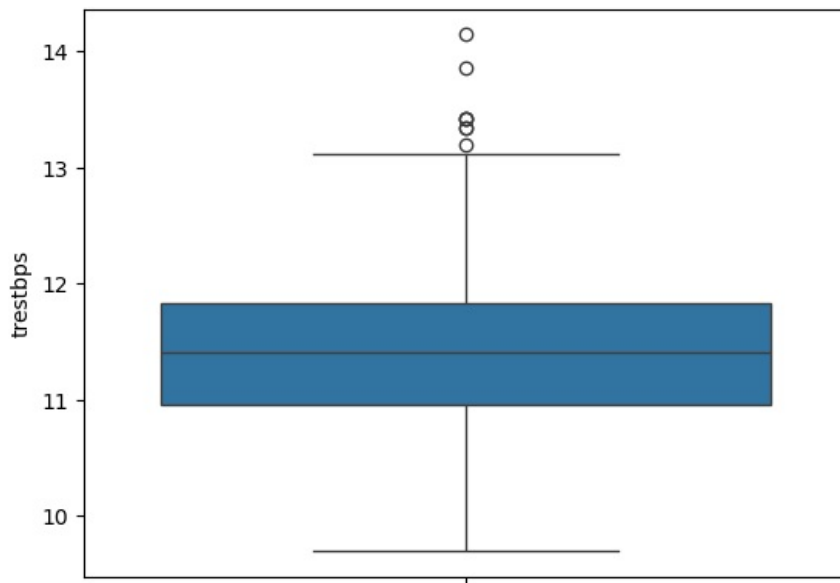
```
In [25]: sns.distplot(data['trestbps'])
```

```
Out[25]: <Axes: xlabel='trestbps', ylabel='Density'>
```

```
In [26]: sns.boxplot(data['trestbps'])
```

```
Out[26]: <Axes: ylabel='trestbps'>
```



```
In [27]: #Validating cholesterol outlier
chol_pred_high = (q3.chol + (1.5 * IQR.chol))
chol_pred_high
```

```
Out[27]: np.float64(369.75)
```

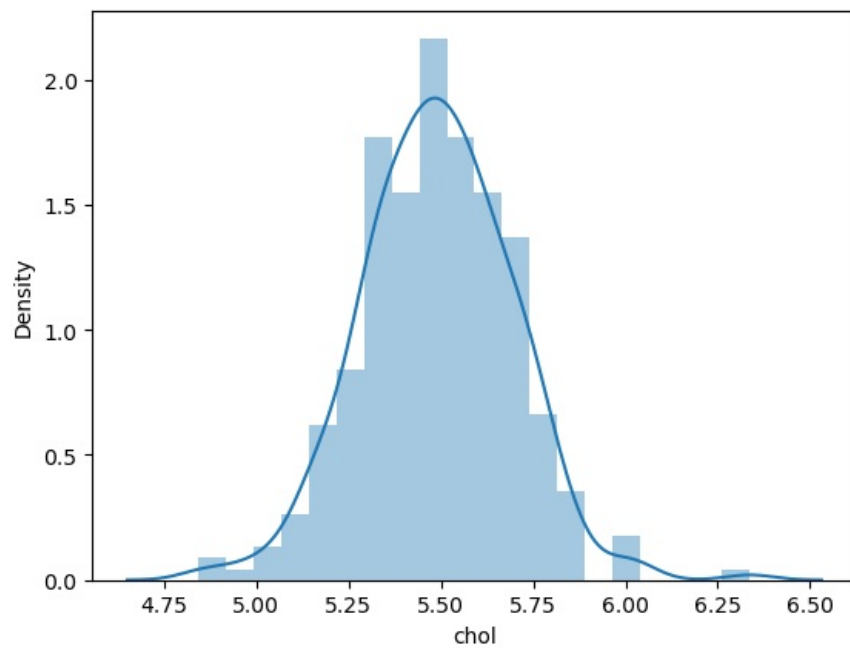
```
In [28]: #Check the indexes which have higher values
index=np.where(data['chol']>chol_pred_high)
index
```

```
Out[28]: (array([ 28,  85,  96, 220, 246]),)
```

```
In [29]: data['chol'] = np.log1p(data['chol'])
```

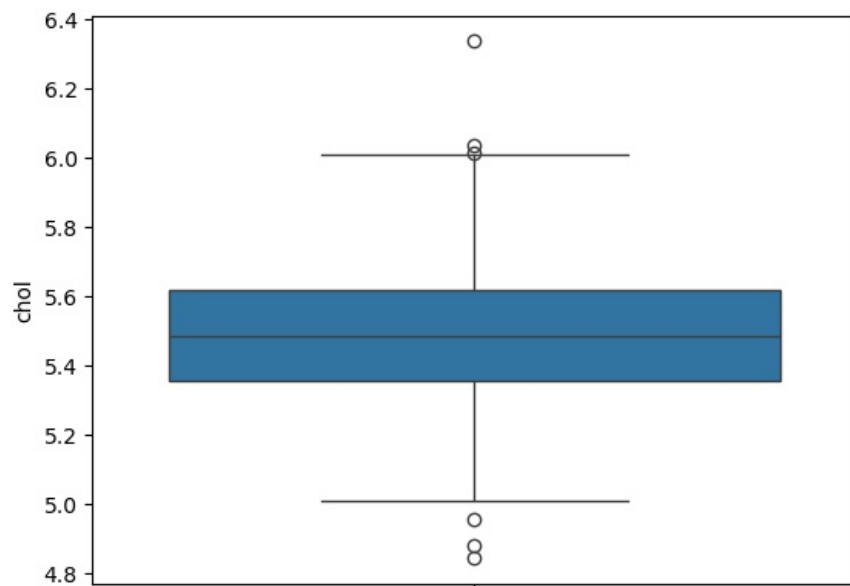
```
In [30]: sns.distplot(data['chol'])
```

```
Out[30]: <Axes: xlabel='chol', ylabel='Density'>
```



```
In [31]: sns.boxplot(data['chol'])
```

```
Out[31]: <Axes: ylabel='chol'>
```



```
In [32]: #Validating oldpeak outlier
op_pred_high = (q3.oldpeak + (1.5 * IQR.oldpeak))
op_pred_high
```

```
Out[32]: np.float64(4.0)
```

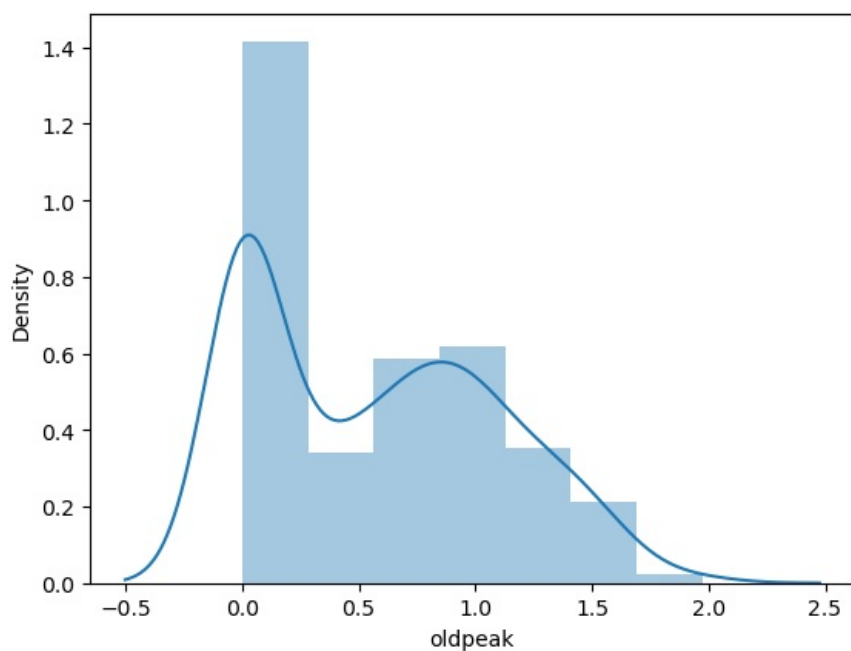
```
In [33]: #Check the indexes which have higher values
index=np.where(data['oldpeak']>op_pred_high)
index
```

```
Out[33]: (array([101, 204, 221, 250, 291]),)
```

```
In [34]: data['oldpeak'] = np.log1p(data['oldpeak'])
```

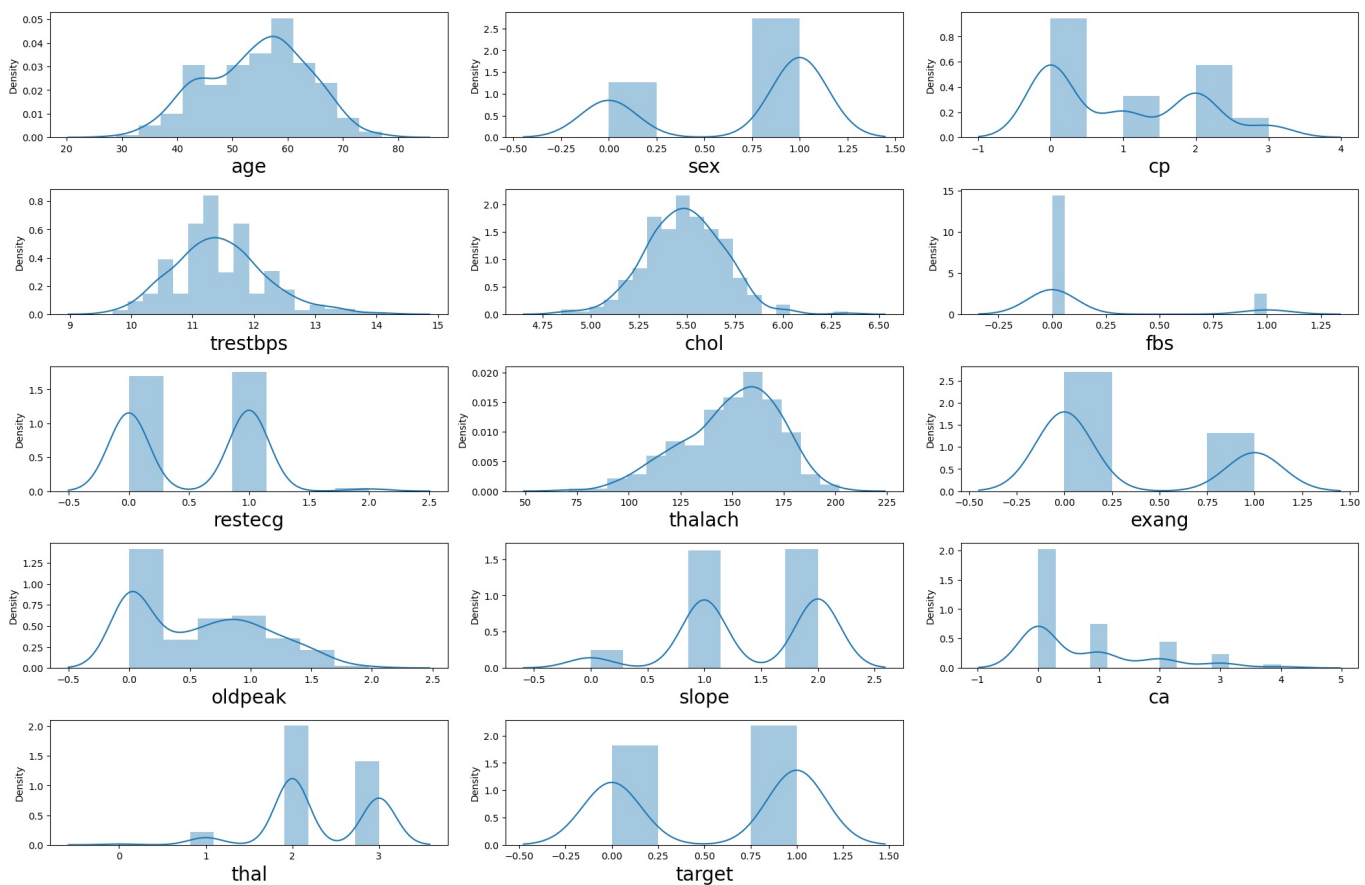
```
In [35]: sns.distplot(data["oldpeak"])
```

```
Out[35]: <Axes: xlabel='oldpeak', ylabel='Density'>
```



In [36]: # Checking the Skewness after treating outliers

```
plt.figure(figsize=(20,15))
plotnumber=1
for column in data:
    if plotnumber<=14:
        ax=plt.subplot(6,3,plotnumber)
        sns.distplot(data[column])
        plt.xlabel(column,fontsize=20)
        plotnumber +=1
plt.tight_layout()
```



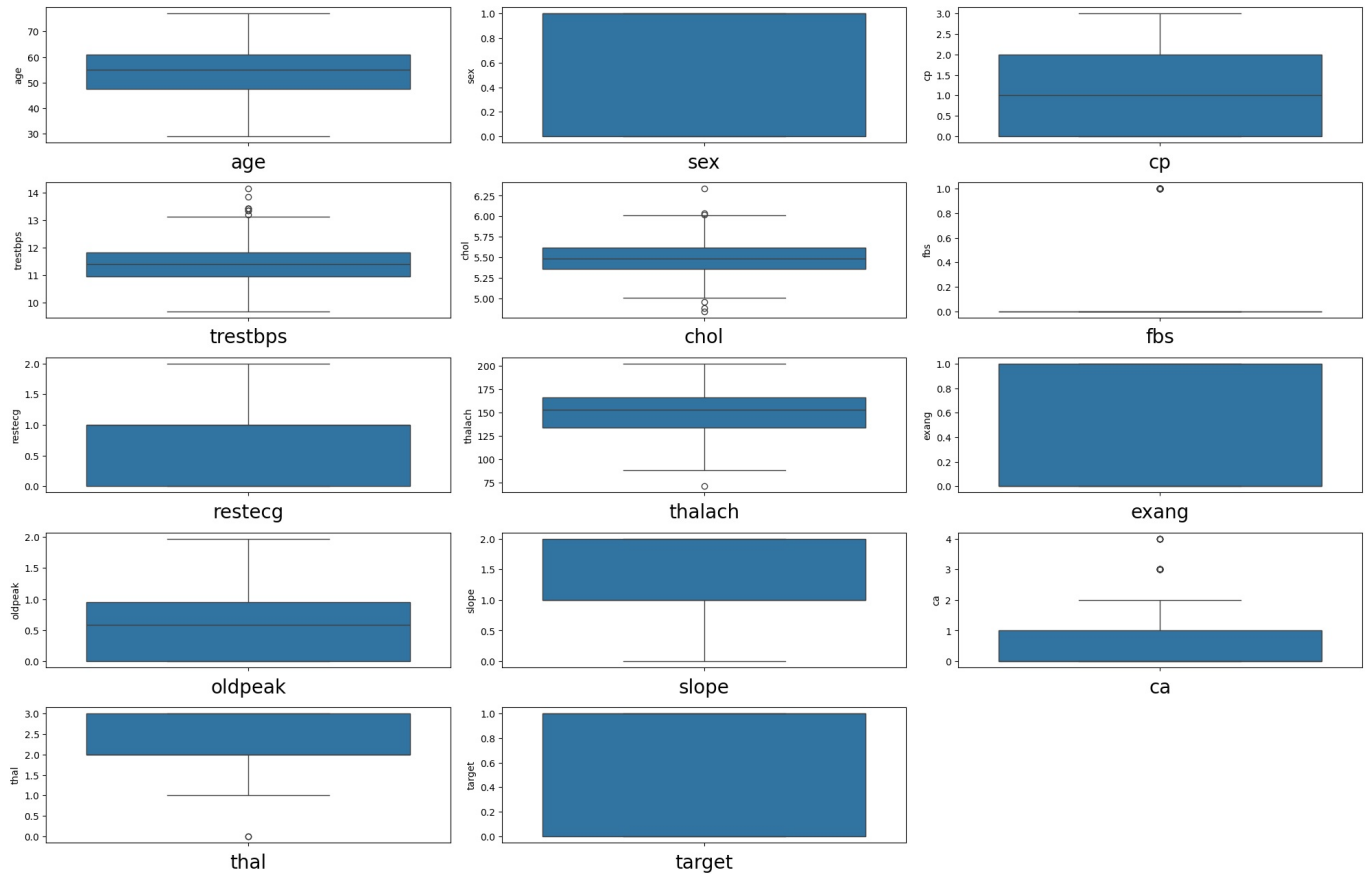
In [37]: # Checking the Skewness after treating outliers

```
plt.figure(figsize=(20,15))
plotnumber=1
```

```

for column in data:
    if plotnumber<=14:
        ax=plt.subplot(6,3,plotnumber)
        sns.boxplot(data[column])
        plt.xlabel(column,fontsize=20)
        plotnumber +=1
plt.tight_layout()

```



- From the distplot and boxplot, we clearly understand the outliers and skewness are treated well.
- For **trestbps** we used **square root transformation**.
- **cholesterol and oldpeak** we used ****Log Transformation****.
- IQR to treat the outliers.

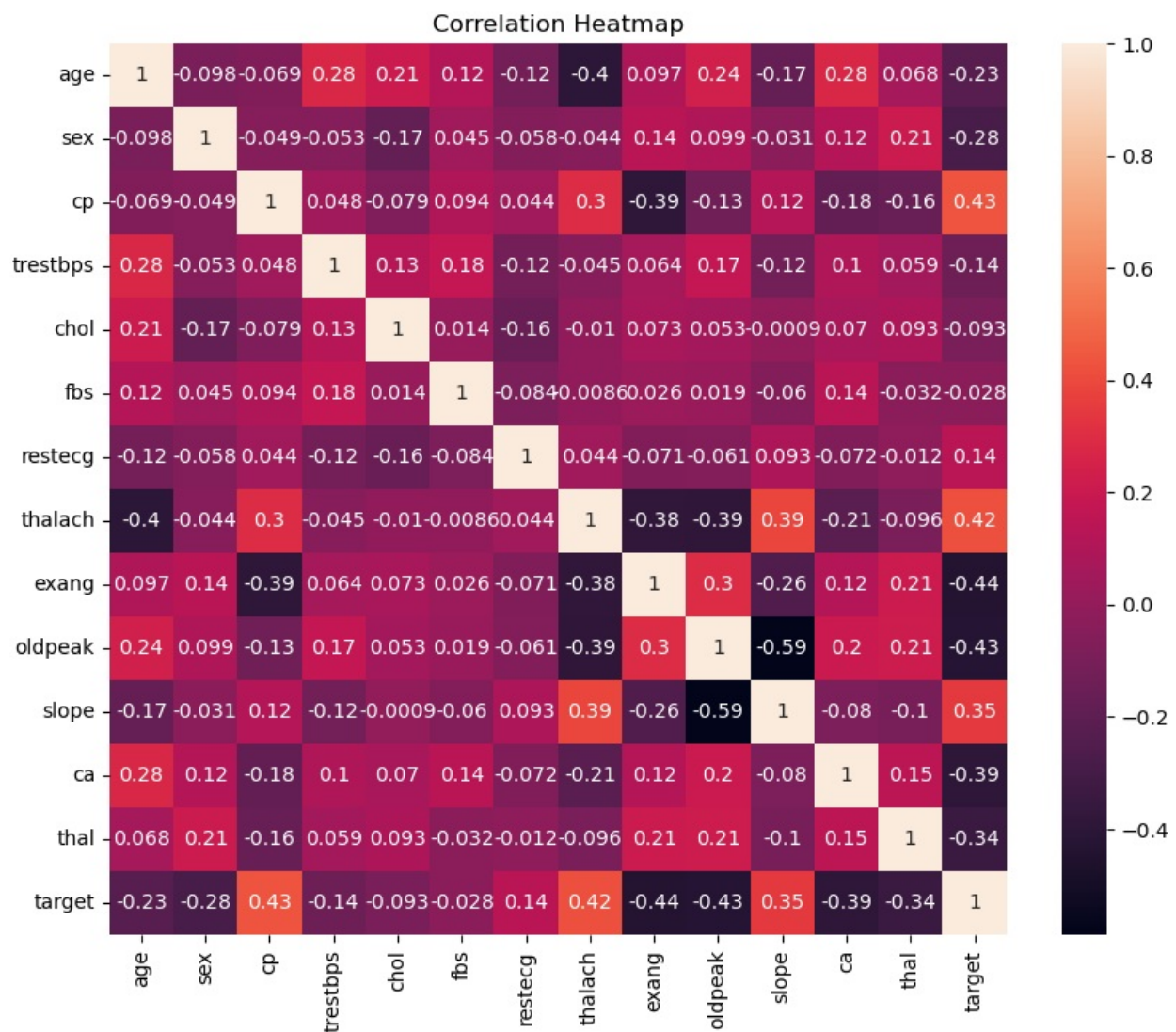
Multicollinearity problem

```
In [38]: data_corr = data.corr()
```

```
In [39]: plt.figure(figsize=(10,8))

sns.heatmap(
    data_corr,
    annot=True,
)

plt.title("Correlation Heatmap")
plt.show()
```



- From the heatmap correlation matrix there are no independent features are correlated with each other.
- The dataset is preprocessed and ready for model training.

```
In [40]: # Saving the dataset in csv
data.to_csv('Cleaned_Heart_Disease_Dataset.csv', index=False)
```

```
In [41]: df = pd.read_csv("Cleaned_Heart_Disease_Dataset.csv")
```

```
In [42]: df.head()
```

```
Out[42]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	63	1	3	12.041595	5.455321	1	0	150	0	1.193922	0	0	1	1
1	37	1	2	11.401754	5.525453	0	1	187	0	1.504077	0	0	2	1
2	41	0	1	11.401754	5.323010	0	0	172	0	0.875469	2	0	2	1
3	56	1	1	10.954451	5.468060	0	1	178	0	0.587787	2	0	2	1
4	57	0	0	10.954451	5.872118	0	1	163	1	0.470004	2	0	2	1